

서울 랜드마크 이미지 분류 모델 구현 및 전이학습 비교 실험

A팀 박소현, 이지현 , 최유경

데이터셋 구조

train	test
: 레이블 있는 데이터 723개 데이터 10개의 클래스	: 레이블 없는 검증 데이터 199개 데이터

```
label
0    67
1    71
2    75
3    71
4    67
5    75
6    75
7    72
8    74
9    76
Name: count, dtype: int64
```

train data를 8:2비율로
train/test 데이터로 분할.

```
train_df = pd.read_csv(train_csv_path)

train_df, val_df = train_test_split(
    train_df,
    test_size=0.2,
    stratify=train_df['label'],
    random_state=42
)

print(len(train_df))
print(len(val_df))
```

578
145

이미지 형식 확인

```
import cv2
image_path = '/content/drive/MyDrive/Synap
img = cv2.imread(image_path, cv2.IMREAD_UN
height, width, channels = img.shape
print(f"이미지 크기: 너비={width}, 높이={h
```

이미지 크기: 너비=960, 높이=540, 채널=3

원본 이미지의 shape는
960*540*3 (W,H,C) -> reshape
224*224로 입력

기본 CNN 모델

```
class BasicBlock(nn.Module):
    def __init__(self, in_channels, out_channels, hidden_dim):
        super().__init__()

        self.Conv1 = nn.Conv2d(in_channels, hidden_dim, kernel_size=3, stride=1, padding=1)
        self.Conv2 = nn.Conv2d(hidden_dim, out_channels, kernel_size=3, stride=1, padding=1)
        self.relu = nn.ReLU()
        self.maxpool = nn.MaxPool2d(kernel_size=2, stride=2)
```

basicBlock

```
def forward(self, x):
```

```
    x = self.Conv1(x)
```

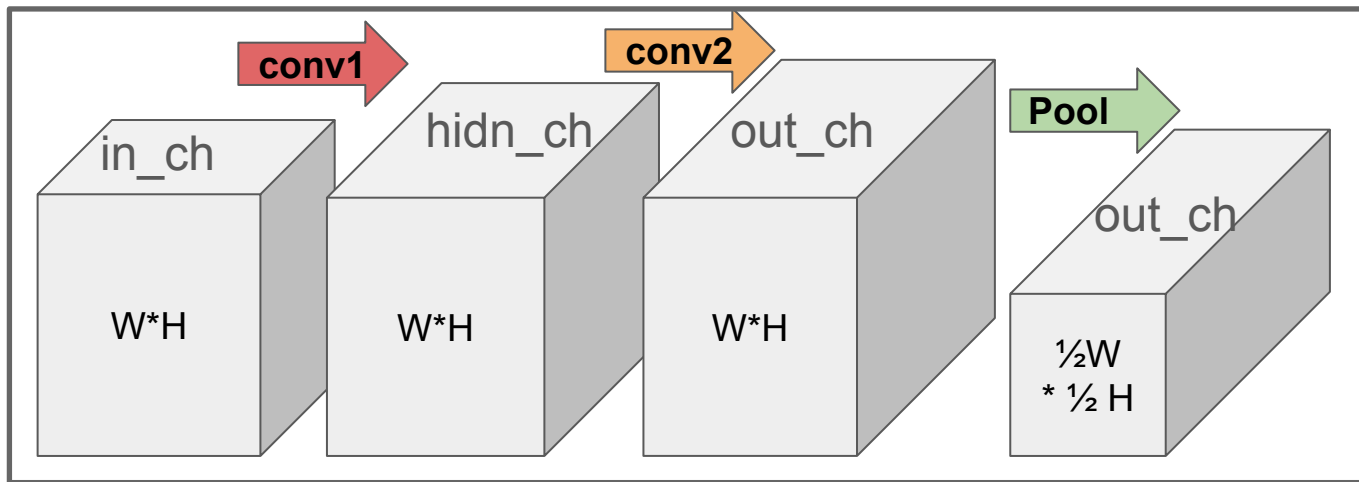
```
    x = self.relu(x)
```

```
    x = self.Conv2(x)
```

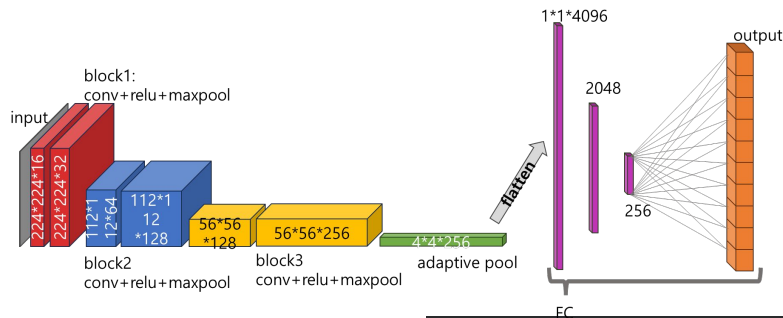
```
    x = self.relu(x)
```

```
    x = self.maxpool(x)
```

```
    return x
```



기본 CNN 모델



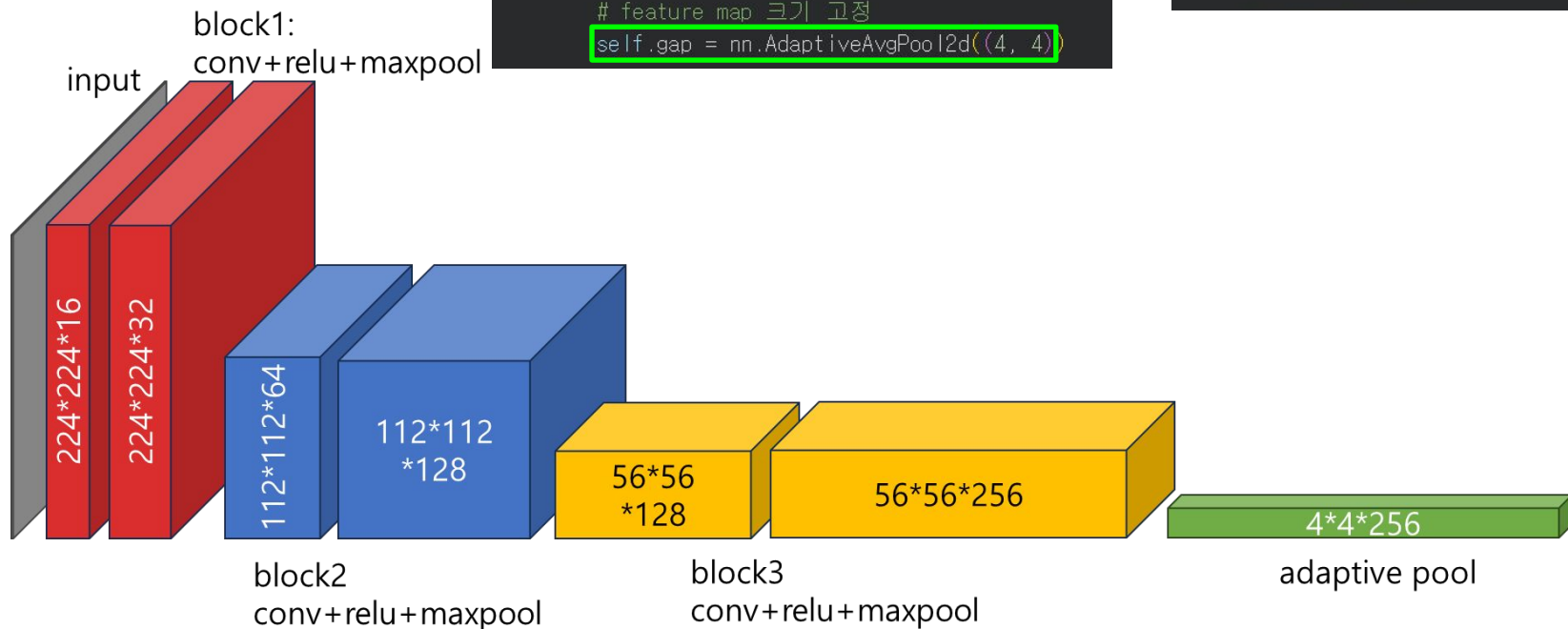
```
class CNN(nn.Module):  
  
    def __init__(self, num_classes):  
  
        super(CNN, self).__init__()  
  
        in, out, hid  
        self.block1 = BasicBlock(3, 32, 16)  
        self.block2 = BasicBlock(32, 128, 64)  
        self.block3 = BasicBlock(128, 256, 128)  
  
        # feature map 크기 고정  
        self.gap = nn.AdaptiveAvgPool2d((4, 4))  
  
        # 256 x 4 x 4 = 4096  
        self.fc1 = nn.Linear(4096, 2048)  
        self.fc2 = nn.Linear(2048, 256)  
        self.fc3 = nn.Linear(256, num_classes)  
  
        self.relu = nn.ReLU()
```

```
def forward(self, x):  
  
    x = self.block1(x)  
    x = self.block2(x)  
    x = self.block3(x)  
  
    # adaptive pooling  
    x = self.gap(x)  
  
    # flatten  
    x = torch.flatten(x, start_dim=1)  
  
    # classifier  
    x = self.fc1(x)  
    x = self.relu(x)  
  
    x = self.fc2(x)  
    x = self.relu(x)  
  
    x = self.fc3(x)  
  
    return x
```

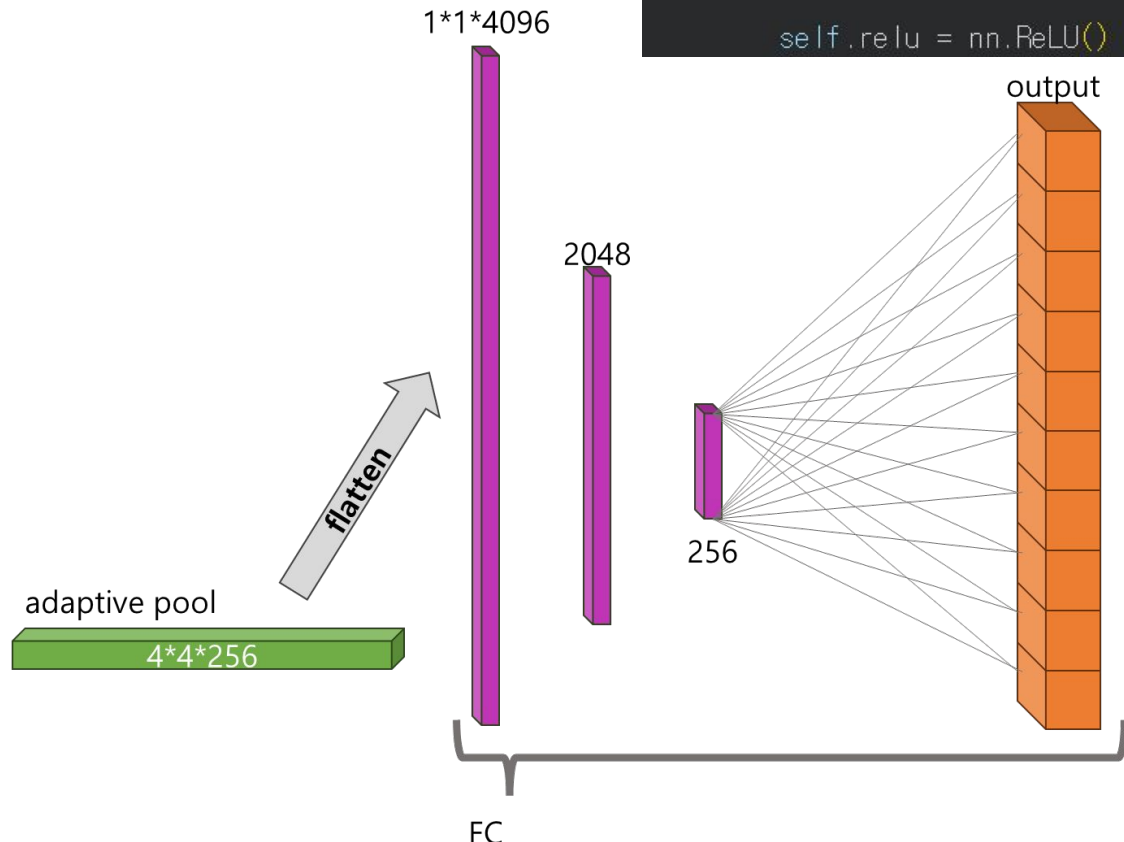
기본 CNN 모델

```
class CNN(nn.Module):  
  
    def __init__(self, num_classes):  
  
        super(CNN, self).__init__()  
  
        in, out, hid  
        self.block1 = BasicBlock(3, 32, 16)  
        self.block2 = BasicBlock(32, 128, 64)  
        self.block3 = BasicBlock(128, 256, 128)  
  
        # feature map 크기 고정  
        self.gap = nn.AdaptiveAvgPool2d((4, 4))
```

```
def forward(self, x):  
  
    x = self.block1(x)  
    x = self.block2(x)  
    x = self.block3(x)  
  
    # adaptive pooling  
    x = self.gap(x)
```



기본 CNN 모델



```
# 256 x 4 x 4 = 4096
self.fc1 = nn.Linear(4096, 2048)
self.fc2 = nn.Linear(2048, 256)
self.fc3 = nn.Linear(256, num_classes)
```

```
self.relu = nn.ReLU()
```

```
def forward(self, x):
```

```
    x = self.block1(x)
    x = self.block2(x)
    x = self.block3(x)
```

```
    # adaptive pooling
    x = self.gap(x)
```

```
    # flatten
    x = torch.flatten(x, start_dim=1)
```

```
    # classifier
    x = self.fc1(x)
    x = self.relu(x)
```

```
    x = self.fc2(x)
    x = self.relu(x)
```

```
    x = self.fc3(x)
```

```
    return x
```

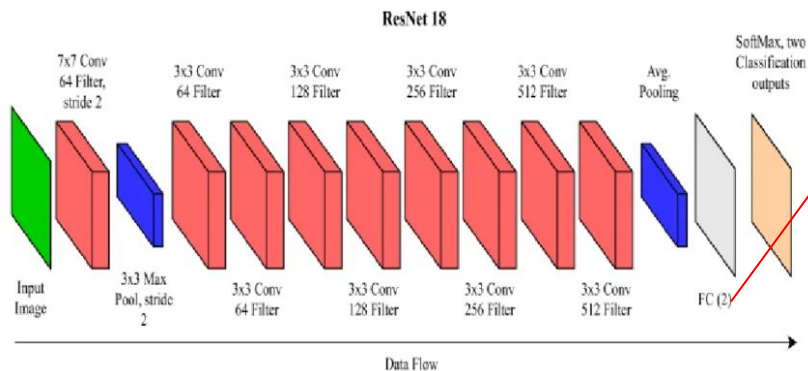
실험

seed 고정하지 않고 학습 실행시, 매번 데이터가 **suffle**되어 학습되므로 같은 모델이더라도 학습마다 **Best Val Acc**가 상이해짐.

- **seed** 고정하는 코드

```
1 import random
2 import numpy as np
3
4 def set_seed(seed=42):
5     random.seed(seed)
6     np.random.seed(seed)
7     torch.manual_seed(seed)
8     torch.cuda.manual_seed(seed)
9     torch.cuda.manual_seed_all(seed)
10    torch.backends.cudnn.deterministic = True
11    torch.backends.cudnn.benchmark = False
12
13 set_seed(42)
```

전이 학습 모델



ResNet18

- 사전 학습 지식 보존
- Fine tuning 전략 수립

```
# 1. 사전 학습된 ResNet18 가중치 로드
transfer_model = models.resnet18(weights=models.ResNet18_Weights.IMAGENET1K_V1)

# 2. 마지막 전결합층(fc)의 입력 노드 수 확인
num_fts = transfer_model.fc.in_features

# 3. 출력 노드를 10개(클래스 수)로 변경하는 새로운 Linear 레이어 설정
transfer_model.fc = nn.Linear(num_fts, 10)

# 4. 모델을 연산 디바이스로 이동
transfer_model = transfer_model.to(device)

# 옵티마이저 설정 (학습률은 Fine-tuning에 적합한 1e-4로 설정)
optimizer = torch.optim.Adam(transfer_model.parameters(), lr=1e-4)

history_tf = train_model(
    model=transfer_model,
    train_loader=train_loader,
    val_loader=val_loader,
    optimizer=optimizer,
    device=device,
    epochs=15,
    save_path=base_path + "/resnet18_transfer.pt"
)
```

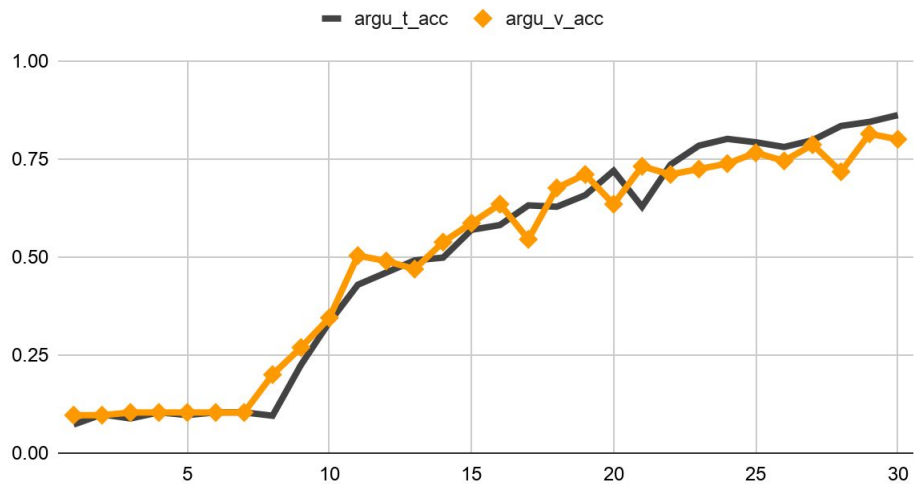

하이퍼 파라미터 실험

- 실험 결과 비교(하이퍼 파라미터 조정) 15epoch

	lr	batch_size	optimizer	best_val_acc
4	0.0001	16	Adam	0.820690
1	0.0010	16	AdamW	0.813793
6	0.0001	32	Adam	0.800000
3	0.0010	32	AdamW	0.793103
7	0.0001	32	AdamW	0.793103
5	0.0001	16	AdamW	0.786207
2	0.0010	32	Adam	0.779310
0	0.0010	16	Adam	0.751724

데이터 증강 실험

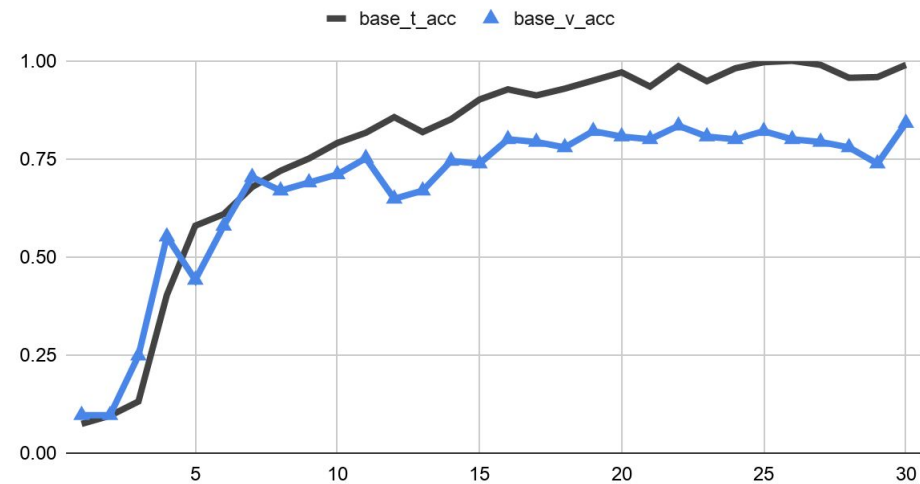
데이터 증강 모델



Base 모델 Loss



베이스 모델



모델 성능 비교

실험 번호	모델	pretrained	freeze 전략	Optimizer	LR	Batch Size	Augmentation	Best Val Acc
0	CNN aseline(15)	x	x	Adam	1e-3	16	x	0.7517
1	CNN hpram1	x	x	AdamW	1e-3	16	x	0.8138
2	CNN hpram2	x	x	Adam	1e-3	32	x	0.7793
3	CNN hpram3	x	x	AdamW	1e-3	32	x	0.7931
4	CNN hpram4	x	x	Adam	1e-4	16	x	0.8207
5	CNN hpram5	x	x	AdamW	1e-4	16	x	0.7862
6	CNN hpram6	x	x	Adam	1e-4	32	x	0.8000
7	CNN hpram7	x	x	AdamW	1e-4	32	x	0.7931
10	transfer	resNet18	x	Adam	1e-4	16	x	1.000
8	CNN baseline(30)	x	x	Adam	1e-3	16	x	0.8414
9	CNN Augmentation	x	x	Adam	1e-3	16	randomHorizontalFlip, RandomRotation(degree : 30)	0.8138

어려웠던 점과 해결 방법

이지현	최유경	박소현
모델 학습 후 성능 도출시 매번, 값이 달라지는 것 때문에 모델의 실제 성능을 파악하기 어려웠습니다. seed 고정 방법을 깨달아 나중에 유용하게 써먹을 수 있을 것 같습니다. 개인적으로 겹치는 코드 부분은 모듈화해 사용하는 것을 선호하는데, 단일 파일 제출인 것도 있고, colab 환경에서도 잘 작동할지 의문이라 코드를 복사해 넣은 것이 약간 아쉽습니다 .	colab 환경으로 협업해보는건 처음이라 환경 세팅이 조금 어려웠습니다. ai에게 물어보고 시도해보니 해결되었습니다.	지현님과 유경님이 보내주시는 코드들이 전부 처음 보는 코드여서, 어떤 코드인지 파악하고 공부하기 위해 .ipynb로 파일을 다운로드한 뒤 AI에게 코드를 설명해달라고 부탁했다. 또, 이렇게 파악한 코드를 기반으로 전이학습 코드를 구현해달라고 요청했다. 다만 여기서 한 가지 아쉬운 점은, 한 학기 동안 이론을 공부했지만 막상 실습에 적용하려니 일대일 매칭이 잘 되지 않아 혼란스럽거나 잘 모르겠는 부분이 많아서, 방학 때 이론 역시 더욱 공부하며 보강해야 겠다는 생각이 들었다.

팀원별 역할

이지현 - A	최유경 - B	박소현 - C
- 데이터 구조 분석 - CSV 처리 - Dataset/DataLoader 구현 - 데이터 증강 실험 - 보고서 1., 2., 3., 5., 6.1),3),4),6) 작성 - 발표자료 작성	- CNN model 구성 - 모델 평가 파이프라인 구축 - 모델 학습 파이프라인 구축 - 하이퍼파라미터 관련 실험 파이프라인 구현 - 보고서 작성 - 발표자료 작성	- 전이 학습 코드 작성 - 전이 학습 관련 실험 진행 - 전이 학습 부분 보고서 작성 - 발표자료 작성